INTEGRAL DRAGON

WEEK 03 · COURSE OVERVIEW

Applied Practice

AI-Accelerated Software Development — Week 3. Five sessions. One repeatable engineering practice.

AI-ACCELERATED SOFTWARE DEVELOPMENT | APPLIED PRACTICE

AI-assisted engineering is most valuable when it helps teams see, reduce, and systematically pay down the friction and technical debt that make software hard to change.

The point is not more code. The point is easier change. A bad codebase plus AI does not become a good codebase — it becomes a faster bad codebase.

The five-part method for the week

STEP 01

Define good

Establish a target state for project health before discussing debt.

→

STEP 02

Identify debt

Name the friction that blocks the target and quantify its interest.

→

STEP 03

Use AI

Inspect, explain, scaffold, and propose targeted improvements.

→

STEP 04

Validate

Prove the change with tests, observability, and security checks.

→

STEP 05

Repeat

Turn one-off cleanup into a standing engineering practice.

From friction to flow. Each session rehearses one step with a working repo and a short, focused exercise.

Five sessions, one per day, sixty minutes each

DAY 01

What Good Looks Like

Define project health. Walk a repo with a scorecard.

DAY 02

Technical Debt as Friction

Classify debt. Estimate the interest it charges change.

DAY 03

Using AI to Inspect & Improve

Prompt patterns for repo analysis, docs, tests, and small refactors.

DAY 04

Validation

Tests, observability, and security as gates for AI-assisted change.

DAY 05

Making It Repeatable

Cadence, ownership, workflow. Build a 30-day plan.

Every day follows the same shape: ten minutes of framing, fifteen of examples, twenty of exercise, ten of debrief, five of next-step setup.

What good looks like — nine health categories

CATEGORY 01

Documentation & Discoverability

Docs are current enough to trust; new engineers find the path from clone to running system.

CATEGORY 02

Architecture Clarity

Boundaries, responsibilities, and dependency direction are legible at a glance.

CATEGORY 03

Reproducible Setup

Any clean machine reaches a working dev state with bounded effort.

CATEGORY 04

Testability & Confidence

Tests help the team move faster. Failing tests are actionable, not brittle.

CATEGORY 05

Build, Deploy & Release

Versions mean something. Deployed artifacts are identifiable and verifiable.

CATEGORY 06

Data & Schema Discipline

Schema changes are reviewable and repeatable. Test data is intentional.

CATEGORY 07

Observability

Logs, traces, and metrics answer what happened, where, when, and why.

CATEGORY 08

Security & Supply Chain

Secrets managed deliberately. Vulnerable dependencies are visible.

CATEGORY 09

Workflow & Collaboration

Small, reviewable changes land consistently. AI diffs are reviewed like any other.

Technical debt, named in nine categories

DEBT 01

Knowledge Debt

Outdated docs, no architecture map, unclear ownership, hidden operational steps.

DEBT 02

Design Debt

Tangled dependencies, mixed responsibilities, duplicated logic, weak boundaries.

DEBT 03

Environment Debt

Snowflake machines, missing setup automation, dependency version chaos.

DEBT 04

Quality-Confidence Debt

Missing tests, flaky tests, meaningless coverage, no regression confidence.

DEBT 05

Release Debt

Manual release steps, unknown deployed version, weak artifact traceability.

DEBT 06

Data & Schema Debt

Untracked schema changes, fragile migrations, impossible test-data setup.

DEBT 07

Observability Debt

Noisy logs, missing correlation, no traces, vanity dashboards.

DEBT 08

Security & Supply Chain

Hardcoded secrets, unreviewed dependencies, no scanning, unclear provenance.

DEBT 09

Workflow Debt

Huge PRs, weak review norms, cleanup never scheduled, AI changes under-reviewed.

Where AI helps — and where it is risky

AI IS STRONG WHEN

- Summarizing a codebase or long scripts
- Finding patterns, duplication, and inconsistencies
- Identifying documentation and test gaps
- Generating first-pass scaffolds and checklists
- Proposing small, scoped refactors
- Tracing cross-cutting logic and dependencies
- Turning issue patterns into cleanup plans

AI IS RISKY WHEN

- Rewriting large parts of a system without context
- Writing tests that assert implementation trivia
- Producing documentation that sounds right but is wrong
- Making security-sensitive changes without review
- Generating more code than the team can absorb
- Normalizing insecure patterns seen in public repos
- Hiding weak reasoning behind fluent summaries

Use AI to inspect, explain, propose, and scaffold. Use humans to decide, validate, and own the change. Treat AI output as draft material with a burden of proof.

The week ends with a 30-day plan, not a rewrite

3	2	1	1
QUICK WINS	STRUCTURAL IMPROVEMENTS	WORKFLOW CHANGE	MEASUREMENT CHANGE

- **Staged modernization beats rewrite fantasies.** Small, validated improvements compound.
- **Improvement needs cadence and ownership.** Debt backlogs live alongside feature work, not after it.
- **AI usage needs policy, not just access.** Review AI diffs with the same rigor as human diffs.

• APPLIED PRACTICE — CLOSE OUT

The point is not more code. The point is easier change.

Good engineering makes AI more useful. If you cannot validate the change, AI only helped you move faster in the dark.